

Get the Values of Environment Variables in Postman

Reading the captured APIID global and using it in a follow-up GET request

Hands-On	13
Scenario	Get the values of environment variables in Postman
Tool	Postman + GoRest API
Method	GET
Endpoint URL	https://gorest.co.in/public/v2/users/{{APIID}}
Auth	Bearer Token (from GoRest profile)
Outcome	Same user fetched back; APIID global resolved into URL
Duration	10 minutes
Prerequisite	Assignment 12 completed (APIID global already set)

Objective

The objective of this assignment is to **read** the global variable APIID that was set by the Create User request in the previous assignment, and use it to fetch the same user back via a GET request. The variable is consumed in two complementary ways: (1) directly in the URL using the `{{APIID}}` templating syntax, and (2) programmatically inside a **pre-request script** using `pm.globals.get("APIID")`. This pattern — reading a previously stored variable to drive a subsequent request — is the foundation of *request chaining* in Postman.

Two Ways to Read a Variable

Postman exposes stored variables (globals, environments, collections, and locals) in two places:

- **Templating in any field** — use the `{{variable}}` syntax inside URLs, headers, query params, request bodies, and authorization fields. Postman replaces the placeholder with the variable's current value just before sending the request.
- **Scripts** — in pre-request and post-response scripts, use the JavaScript API: `pm.globals.get("name")`, `pm.environment.get("name")`, `pm.collectionVariables.get("name")`, or the unified `pm.variables.get("name")` which searches all scopes in priority order.

Steps

1 Go to the Right Workspace

Open Postman and sign in if not already logged in.

Click the **Workspaces** dropdown and select the workspace used in the previous assignment.

Confirm the APIID global variable is still set (click the **Eye** icon in the top-right and check the **Globals** section). If it has been cleared, run the previous POST Create User request first to repopulate it.

2 Create a New Request for GET User

Inside the right collection, click the **+** button or **Add a request** link.

Rename the request to something descriptive such as *Get User By ID* or *GET GoRest User*.

Save with **Ctrl + S** / **Cmd + S**.

3 Set Method and URL Using the {{APIID}} Variable

Set the HTTP method to **GET**.

Enter the URL with the variable in place of the user id:

https://gorest.co.in/public/v2/users/{{APIID}}

Postman colours **{{APIID}}** in orange when the variable exists. Hovering over it shows the **scope** (GLOBAL / ENVIRONMENT / COLLECTION) and the **current value** that will be substituted at send time.

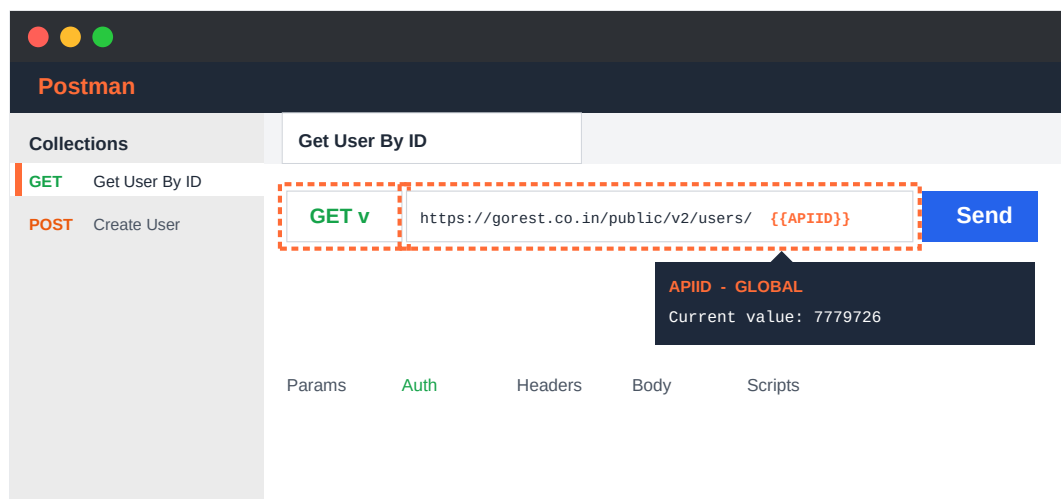


Figure 1 — URL bar showing **{{APIID}}** in orange. The tooltip preview reveals the variable scope (**GLOBAL**) and the value (**7779726**) Postman will substitute when the request is sent.

4 Add a Pre-Request Script to Read the Variable

Click the **Scripts** tab below the URL bar.

Make sure the **Pre-request** sub-tab is selected (this code runs *before* the request is sent).

Paste the JavaScript shown in the **Pre-Request Script** section below.

The script reads the APIID global with `pm.globals.get`, prints its value to the Postman **Console** for verification, and copies it into an environment variable `currentUserId` for demonstration.

Open the Postman **Console** (bottom-left of the workbench, or **View** → **Show Postman Console**) before sending so the logged value can be inspected after the run.

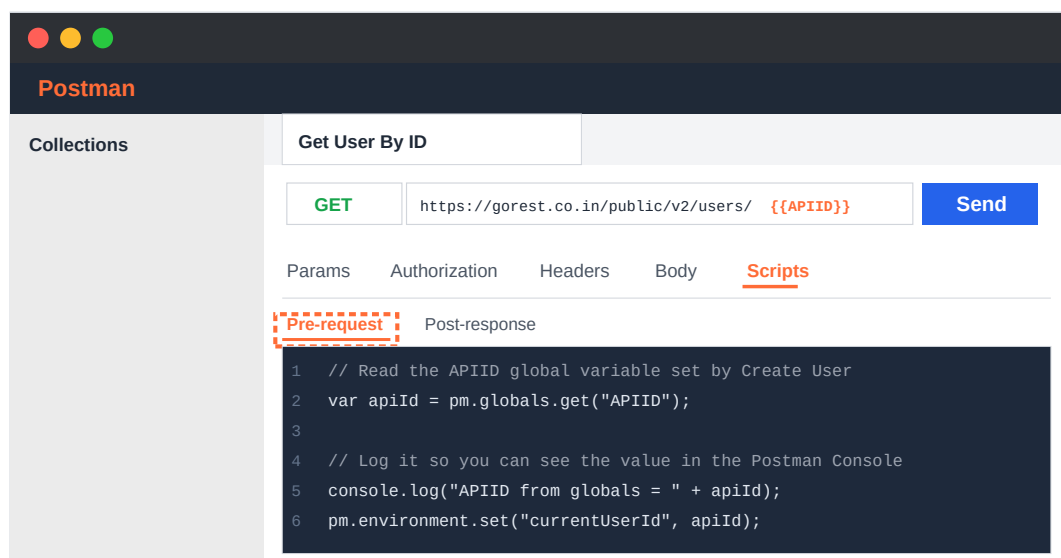


Figure 2 — **Scripts** tab on the **Pre-request** sub-tab with JavaScript that reads the **APIID** global and logs it to the Postman Console.

Pre-Request Script (Input)

Paste this JavaScript into the **Pre-request** editor:

```
// Read the APIID global variable set by Create User
var apiId = pm.globals.get("APIID");

// Log it so you can see the value in the Postman Console
console.log("APIID from globals = " + apiId);
pm.environment.set("currentUserId", apiId);
```

5 Configure Authorization (Bearer Token)

Click the **Authorization** tab.

From the **Type** dropdown, choose **Bearer Token**.

Paste the same GoRest access token used in the previous assignment (from the GoRest profile / Access Tokens page) into the **Token** field.

Tip: If the token is reused frequently, save it as an environment variable (e.g. `gorest_token`) and reference it here as `{{gorest_token}}` — that way it never needs to be pasted again.

6 Click Send and Verify the Status Code

Click the blue **Send** button.

Confirm the **Status** indicator shows **200 OK** in green.

If the status is **404 Not Found**, the user with that id no longer exists — GoRest data resets daily, so re-run the previous Create User request to mint a fresh user (and a fresh APIID), then retry this GET.

7 Verify the Response Body

Inspect the **Body** tab of the response panel.

The JSON should match the user that was created in the previous assignment:

- **id** — the same numeric id stored in the APIID global.
- **name, email, gender, status** — the same values that were sent in the POST body.

Open the **Postman Console** at the bottom and confirm that the pre-request log line is visible:

```
APIID from globals = 7779726
```

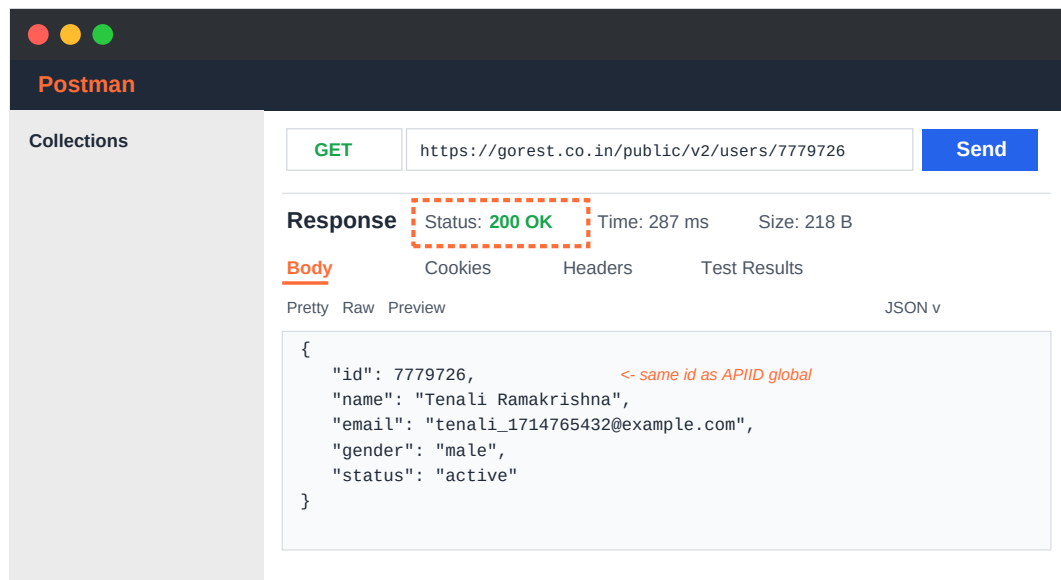


Figure 3 — Response panel showing **200 OK** and the same user record that was created in the previous assignment. The **id** field matches the value stored in the APIID global.

Expected Response

On success, GoRest returns **HTTP 200 OK** with the user record:

```
{
  "id": 7779726,
  "name": "Tenali Ramakrishna",
  "email": "tenali_1714765432@example.com",
  "gender": "male",
  "status": "active"
}
```

The exact **id**, **email**, and **field** values will match whatever was created in the previous assignment. As long as the response **id** equals the **APIID** global, the variable was correctly read and used.

Verification Checklist

- 1 Response status is **200 OK**.
- 2 Response body contains the **same user record** that was created in the previous assignment (matching **id**, **name**, **email**, **gender**, **status**).
- 3 Hovering over `{{APIID}}` in the URL bar shows the resolved value matching the response **id**.
- 4 The Postman Console shows the log line `APIID from globals = <id>` from the pre-request script.
- 5 Request is saved (**Ctrl + S** / **Cmd + S**) inside the collection.

Conclusion

The **APIID** global variable saved in the previous assignment was successfully read and used to fetch the same user back. Two complementary techniques were demonstrated — `{{APIID}}` templating in the URL, and `pm.globals.get("APIID")` in a pre-request script. Combined with the post-response capture from the previous assignment, this completes the basic **request-chaining** pattern: capture a

value from one response, store it in a variable, and consume it in subsequent requests — the same pattern used for tokens, session ids, and any other dynamic data that needs to flow through an API workflow.